CSCE 411: Design and Analysis of Algorithms

Lecture 2: Fast Fourier Transforms

Date: January 15, 2026 *Nate Veldt, updated by Samson Zhou*

Course Logistics

- Read Chapter 30 of the textbook for this week's material on polynomials and the FFT.
- Syllabus quiz is due Sat, Jan 17. HW 1 and intro video due Fri, Jan 23.

1 Motivation: Polynomial Multiplication

We will introduce a powerful application of the divide and conquer paradigm: the Fast Fourier Transform (FFT). We will motivate it through the problem of polynomial multiplication.

Definition 1. A _____ in the variable x over an algebraic field is a representation of a function $A(x)$ as a formal sum:

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

The values $a_0, a_1, \ldots, a_{n-1}$ are the _____ of the polynomial.

Example 1. Polynomial Multiplication

Input: Two polynomials $A(x) = \sum_{j=0}^{n-1} a_j x^j$ and $B(x) = \sum_{j=0}^{n-1} b_j x^j$ of degree bound n.

Output: A polynomial $C(x) = A(x)B(x)$ of degree bound $2n-1$ such that:

$$C(x) = \sum_{j=0}^{2n-2} c_j x^j \quad \text{where} \quad c_j = \sum_{k=0}^{j} a_k b_{j-k}$$

An instance of this problem is multiplying $A(x) = 2 + 3x$ and $B(x) = 1 - x$.

Question 1. What is the asymptotic runtime of multiplying two polynomials of degree bound n using the standard, naive approach?

A $O(1)$

B $O(n)$

C $O(n \log n)$

D $O(n^2)$

E Other

2 Representations of Polynomials

To multiply polynomials faster, we can change how we represent them.

2.1 Coefficient Representation

We represent $A(x)$ as a vector of coefficients $a = (a_0, a_1, \ldots, a_{n-1})$.

- Evaluating $A(x)$ at a given point takes _____ time.
- Multiplying two polynomials takes _____ time.

2.2 Point-Value Representation

A polynomial of degree bound n is uniquely determined by its values at n distinct points. We can represent $A(x)$ as a set of n point-value pairs:

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$$

where $y_k = A(x_k)$ for $k = 0, 1, \ldots, n-1$.

Why use point-value representation?
If we have A and B in point-value form evaluated at the *same* points $x_0, \ldots, x_{2n-1}$, how long does it take to compute $C(x) = A(x)B(x)$ in point-value form?

Question 2. How many point-value pairs do we need to compute the polynomial $C(x) = A(x)B(x)$ uniquely, assuming A and B have degree bound n?

A n

B $2n - 1$

C n^2

D $2n$

The Strategy:

- **Evaluate**: Convert A and B to point-value form at $2n$ points.
- **Point-wise Multiply**: Multiply the values in _____ time.

- **Interpolate**: Convert C back to coefficient form.

If we use arbitrary points, Evaluation and Interpolation take _____ time. We need to choose our evaluation points strategically!

3 Background Math: Complex Roots of Unity

To evaluate our polynomials efficiently, we will evaluate them at the **complex roots of unity**.

Definition 2. The **n-th complex roots of unity** are the n complex numbers ω such that:

Using Euler's formula, $e^{iu} = \cos(u) + i\sin(u)$, these roots are given by:

$$\omega_n^k = e^{2\pi ik/n} \quad \text{for } k = 0, 1, \ldots, n-1$$

Example 2. Elementary Example: $n = 4$
The 4-th roots of unity are:

- $k = 0 : \omega_4^0 =$ ___
- $k = 1 : \omega_4^1 =$ __
- $k = 2 : \omega_4^2 =$ _____
- $k = 3 : \omega_4^3 =$ _____

3.1 Crucial Properties for Divide and Conquer

Lemma 3.1. ***Cancellation Lemma:*** *For any integers $n \geq 0, k \geq 0, d > 0$:*

Proof:
By the definition of the complex roots of unity, we have:

This allows us to mathematically "cancel" common factors in the exponent's fraction.

Lemma 3.2. ***Halving Lemma:*** *If $n > 0$ is even, then the squares of the n complex n-th roots of unity are exactly the $n/2$ complex $(n/2)$-th roots of unity.*

Proof:
Applying the Cancellation Lemma with $d = 2$, we immediately get $(\omega_n^k)^2 = \omega_n^{2k} = \omega_{n/2}^k$. However, we must also show that squaring all n roots of unity yields exactly the $n/2$ distinct $(n/2)$-th roots of unity without missing any. For $k \geq n/2$, we can write $k = j + n/2$ where $j < n/2$. Then:

This shows that squaring ω_n^j and squaring $\omega_n^{j+n/2}$ produce the exact same result!

The Halving Lemma implies that if we square our n evaluation points, we are left with only _____. This is perfect for divide and conquer!

3.2 The Summation Lemma

Before we move to the algorithm, we introduce a third lemma that will be strictly necessary when we want to reverse the process during Interpolation.

Lemma 3.3. ***Summation Lemma:*** *For any integer $n \geq 1$ and non-zero integer k not divisible by n:*

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = \_\_$$

Proof:
This is a standard finite geometric series with common ratio $r = \omega_n^k$. Recall the formula for the sum of a finite geometric series: $\sum_{j=0}^{n-1} r^j = \frac{1-r^n}{1-r}$.
Substituting $r = \omega_n^k$, we get:

Because k is not a multiple of n, we know $\omega_n^k \neq 1$, which guarantees the denominator is not zero. Geometrically, this means the vector sum of any regular polygon centered at the origin in the complex plane perfectly cancels out.

4 Intuition for the Fast Fourier Transform

We want to evaluate a polynomial $A(x)$ of degree bound n at the n-th roots of unity: $1, \omega_n, \omega_n^2, \ldots, \omega_n^{n-1}$. Assume n is a power of 2.

Divide We can split $A(x)$ into its even-indexed coefficients and odd-indexed coefficients:

$$A^{[0]}(x) = a_0 + a_2 x + a_4 x^2 + \ldots + a_{n-2} x^{n/2-1}$$
$$A^{[1]}(x) = a_1 + a_3 x + a_5 x^2 + \ldots + a_{n-1} x^{n/2-1}$$

We can express $A(x)$ in terms of these new polynomials:

$$A(x) = \_\_\_\_\_$$

Conquer To evaluate $A(x)$ at the n roots of unity, we need to evaluate $A^{[0]}$ and $A^{[1]}$ at the *squares* of the n roots of unity.

By the **Halving Lemma**, these squares are exactly the _____.

So we just need to evaluate two polynomials of degree bound $n/2$ at $n/2$ roots of unity.
This is two recursive calls of size $n/2$!

Combine For $k < n/2$, we compute:

$$A(\omega_n^k) = A^{[0]}(\omega_{n/2}^k) + \omega_n^k A^{[1]}(\omega_{n/2}^k)$$

For $k \geq n/2$, we use the fact that $\omega_n^{k+n/2} = -\omega_n^k$:

$$A(\omega_n^{k+n/2}) = \_\_\_\_\_$$

5 The Algorithm

Here is the pseudocode for the recursive FFT algorithm, which evaluates a polynomial at the n-th roots of unity.

Algorithm 1 Recursive-FFT(a)

$n = \textbf{length}(a)$
if $n == 1$ **then**
    Return a
end if
$\omega_n = e^{2\pi i/n}$
$\omega = 1$
$a^{[0]} = (a_0, a_2, \ldots, a_{n-2})$
$a^{[1]} = (a_1, a_3, \ldots, a_{n-1})$
$y^{[0]} = \text{RECURSIVE-FFT}(a^{[0]})$
$y^{[1]} = \text{RECURSIVE-FFT}(a^{[1]})$
Let y be a new array of length n
for $k = 0$ to $n/2 - 1$ **do**
    $y_k = y_k^{[0]} + \omega \cdot y_k^{[1]}$
    $y_{k+n/2} = y_k^{[0]} - \omega \cdot y_k^{[1]}$
    $\omega = \omega \cdot \omega_n$
end for
Return y

6 Formal Analysis

Question 3. What is the recurrence relation for the runtime $T(n)$ of the RECURSIVE-FFT algorithm?

A $T(n) = T(n/2) + \Theta(n)$

B $T(n) = 2T(n/2) + \Theta(1)$

C $T(n) = 2T(n/2) + \Theta(n)$

D $T(n) = 4T(n/2) + \Theta(n)$

Using the **Master Theorem** from Lecture 1:

- $a =$ __
- $b =$ __

- $f(n) =$ _____

Since $f(n) = \Theta(n^{\log_b a})$, we are in Case 2. Thus, the runtime is:

$$T(n) = \_\_\_\_\_$$

7 Tracing the Recursive FFT

To solidify our understanding, let's manually trace the RECURSIVE-FFT algorithm on a small example.
Let $A(x) = 1 + 2x + 3x^2 + 4x^3$. We want to evaluate this at the 4-th roots of unity ($n = 4$). Our input coefficient vector is $a = (1, 2, 3, 4)$. The 4-th roots of unity are $1, i, -1, -i$.

Step 1: Top-Level Divide ($n = 4$) We split the initial vector a into even and odd indices:

- $a^{[0]} =$ _____
- $a^{[1]} =$ _____

We now make two recursive calls: RECURSIVE-FFT($a^{[0]}$) and RECURSIVE-FFT($a^{[1]}$).

Step 2: Recursive Calls ($n = 2$) For the recursive calls where $n = 2$, the 2-nd roots of unity are $\omega_2^0 = 1$ and $\omega_2^1 = -1$. Because $n = 2$, the algorithm will recursively split again into size 1 (the base case) which simply returns the scalar coefficient.
Let's compute the results of evaluating $A^{[0]}$ and $A^{[1]}$ at 1 and -1:
Evaluating $A^{[0]}(x) = 1 + 3x$:

- $y_0^{[0]} = A^{[0]}(1) = 1 + 3(1) =$ ___
- $y_1^{[0]} = A^{[0]}(-1) = 1 + 3(-1) =$ _____

Evaluating $A^{[1]}(x) = 2 + 4x$:

- $y_0^{[1]} = A^{[1]}(1) = 2 + 4(1) =$ ___
- $y_1^{[1]} = A^{[1]}(-1) = 2 + 4(-1) =$ _____

Step 3: The Combine Step ($n = 4$) We are now back at the top level. The principal root of unity for $n = 4$ is $\omega_4 = i$. Our formula for the combine step loops k from 0 to 1 (which is $n/2 - 1$):

$$y_k = y_k^{[0]} + \omega_4^k \cdot y_k^{[1]} \quad \text{and} \quad y_{k+2} = y_k^{[0]} - \omega_4^k \cdot y_k^{[1]}$$

For $k = 0$ ($\omega = \omega_4^0 = 1$):

- $y_0 = y_0^{[0]} + (1) \cdot y_0^{[1]} = 4 + 6 =$ ____

- $y_2 = y_0^{[0]} - (1) \cdot y_0^{[1]} = 4 - 6 =$ _____

For $k = 1$ ($\omega = \omega_4^1 = i$):

- $y_1 = y_1^{[0]} + (i) \cdot y_1^{[1]} = -2 + (i)(-2) =$ _____
- $y_3 = y_1^{[0]} - (i) \cdot y_1^{[1]} = -2 - (i)(-2) =$ _____

Final Result: Evaluated at $(1, i, -1, -i)$, the polynomial yields the point-value vector:

$$y = \_\_\_\_\_$$

Sanity check: Let's manually evaluate $A(x)$ at $x = i$:

$$A(i) = 1 + 2(i) + 3(i^2) + 4(i^3) = 1 + 2i - 3 - 4i = \_\_\_\_\_$$

Matches perfectly!

8 The Matrix Perspective (Discrete Fourier Transform)

When we evaluate a polynomial $A(x)$ at the n-th roots of unity, we are fundamentally applying a linear transformation to its coefficient vector. We can write this elegantly as a matrix-vector product $y = V_n a$, where a is the vector of coefficients, y is the vector of evaluations, and V_n is a special matrix called the **Vandermonde Matrix**.

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \cdots & \omega_n^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \cdots & \omega_n^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

Notice that the entry in row j and column k (0-indexed) is given by:

$$(V_n)_{j,k} = \_\_\_\_\_$$

This specific transformation is known as the **Discrete Fourier Transform (DFT)**.

Example 3. The 4×4 DFT Matrix

For $n = 4$, substituting the principal root $\omega_4 = i$, the matrix V_4 looks like this:

$$V_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & i^2 & i^3 \\ 1 & i^2 & i^4 & i^6 \\ 1 & i^3 & i^6 & i^9 \end{bmatrix} = \_\_\_\_\_$$

Question 4. What is the naive runtime of computing this matrix-vector product using standard matrix multiplication?

A $O(n)$

B $O(n \log n)$

C $O(n^2)$

D $O(n^3)$

The Fast Fourier Transform is simply an extraordinarily efficient algorithm to multiply this highly structured matrix V_n by a vector in $O(n \log n)$ time, bypassing the naive $O(n^2)$ matrix-vector multiplication bound!

9 Interpolation and the Inverse FFT

To convert our point-value results back into coefficient form, we need to invert this process. In matrix terms, we know $y = V_n a$. To solve for a, we must compute:

$$a = \_\_\_\_\_$$

Inverting an arbitrary $n \times n$ matrix takes $O(n^3)$ time. Fortunately, the Vandermonde matrix of the roots of unity has a beautifully simple inverse, which we can prove using the **Summation Lemma**.

Theorem 9.1. *The entries of the inverse matrix V_n^{-1} are given by:*

$$(V_n^{-1})_{j,k} = \frac{1}{n}\omega_n^{-jk}$$

Let's prove this! If this is the true inverse, then multiplying V_n^{-1} by V_n should yield the Identity matrix I_n.
Consider the entry in row j and column k of the product matrix $M = V_n^{-1}V_n$. This is the dot product of the j-th row of V_n^{-1} and the k-th column of V_n:

$$M_{j,k} = \sum_{m=0}^{n-1} (V_n^{-1})_{j,m}(V_n)_{m,k} = \sum_{m=0}^{n-1} \left(\frac{1}{n}\omega_n^{-jm}\right)\left(\omega_n^{mk}\right) = \frac{1}{n}\sum_{m=0}^{n-1} \omega_n^{m(k-j)}$$

Case 1: On the Diagonal ($j = k$) If $j = k$, then $k - j = 0$. The exponent becomes 0, and $\omega_n^0 = 1$.

$$M_{j,j} = \frac{1}{n}\sum_{m=0}^{n-1} 1 = \frac{1}{n}(n) = \_\_$$

This correctly matches the 1s on the diagonal of the identity matrix!

Case 2: Off the Diagonal ($j \neq k$) If $j \neq k$, then $(k - j)$ is an integer strictly between $-n$ and n, and it is not zero. Thus, it is not divisible by n. By our **Summation Lemma**, the sum of these roots of unity is exactly __!

$$M_{j,k} = \frac{1}{n}(0) = \_\_$$

This correctly matches the 0s off the diagonal of the identity matrix!

What does this mean for our algorithm?

Look closely at the resulting formula to compute the coefficients a_j:

$$a_j = \frac{1}{n} \sum_{k=0}^{n-1} y_k (\omega_n^{-1})^{jk}$$

This is *exactly* the same summation structure as the forward DFT! The only differences are swapping ω_n for ω_n^{-1} and dividing by n. This leads us perfectly to our main theorem.

10 Main Takeaway: The Convolution Theorem

We showed how to evaluate polynomials at roots of unity in $\Theta(n \log n)$ time. To finish polynomial multiplication, we need to interpolate the result back to coefficient form.

Interpolation It turns out that interpolating at the complex roots of unity is mathematically identical to evaluating, but using ω_n^{-1} instead of ω_n, and dividing the final results by n.
Thus, the **Inverse FFT** can also be computed in _____ time!

Fast Polynomial Multiplication Given two polynomials $A(x)$ and $B(x)$ of degree bound n:

1. **Pad:** Add zeros to A and B so they have degree bound $2n$.

2. **Evaluate (FFT):** Compute point-value representations of A and B of length $2n$.
   Runtime: _____

3. **Point-wise Multiply:** Compute $y_k = A(\omega_{2n}^k)B(\omega_{2n}^k)$ for all k.
   Runtime: _____

4. **Interpolate (Inverse FFT):** Convert point-values y_k back to coefficients.
   Runtime: _____

Total Runtime for Polynomial Multiplication: _____.

By leveraging the **Divide and Conquer** paradigm and evaluating at mathematically symmetric points, we can bypass the naive $O(n^2)$ limit. FFT is an incredibly important algorithm used heavily in signal processing, image compression, and fast large-integer multiplication!

11 Convolution and Padding

We have established the complete algorithm for fast polynomial multiplication. But why did we emphasize padding the polynomials to degree bound $2n$ in Step 1?
In mathematics, multiplying two polynomials corresponds to the **Linear Convolution** of their coefficient vectors. If a and b are vectors of length n, their linear convolution $c = a * b$ is a vector of length $2n - 1$.

Question 5. What happens if we skip the padding step? What if we run the FFT on vectors a and b of length n, multiply them pointwise, and run an Inverse FFT of size n?

A The program will crash.

B It will output the first n coefficients of the correct answer.

C It will compute the Cyclic Convolution of the vectors.

D It will run in $O(n^2)$ time instead.

Cyclic Convolution If we do not pad, the FFT operates purely in the modular arithmetic of n-th roots of unity. The operation computes the polynomial multiplication modulo $(x^n - 1)$:

$$C(x) = A(x)B(x) \pmod{x^n - 1}$$

Because $x^n \equiv 1 \pmod{x^n - 1}$, any term in the correct answer of degree n or higher will "wrap around" and be added to the lower degree terms! This phenomenon is known in signal processing as **aliasing**.
For example, if the true polynomial product is $c_0 + c_1x + c_2x^2 + c_3x^3 + c_4x^4$, but we only used an FFT of size $n = 3$, the x^3 term wraps around and adds to the x^0 term, and the x^4 term wraps around and adds to the x^1 term.

By zero-padding our input vectors to length $2n$ (or the nearest power of 2 greater than $2n-1$), we ensure that the maximum possible degree of the product is strictly less than the wrap-around threshold. Thus, the cyclic convolution perfectly matches the desired linear convolution!

12 Practical Implementation: The Iterative FFT

The RECURSIVE-FFT algorithm takes $\Theta(n \log n)$ time, which is asymptotically optimal. However, in practice, making recursive function calls and allocating new arrays for $a^{[0]}$ and $a^{[1]}$ at every level adds a massive constant factor overhead.
Highly optimized libraries (like FFTW, the "Fastest Fourier Transform in the West") compute the FFT **iteratively** and **in-place**.
To see how to do this, let's track where the elements of an array end up at the base cases of the recursion tree. Consider an input array of size $n = 8$ with indices $0, 1, 2, 3, 4, 5, 6, 7$.

- **Level 0 (Start):** $(0, 1, 2, 3, 4, 5, 6, 7)$
- **Level 1 (Evens/Odds):** $(0, 2, 4, 6)$ and $(1, 3, 5, 7)$
- **Level 2 (Evens/Odds):** $(0, 4), (2, 6)$ and $(1, 5), (3, 7)$
- **Level 3 (Leaves):** $0, 4, 2, 6, 1, 5, 3, 7$

Let's look at the binary representations of these indices:

Original Index	**Binary**	**Leaf Index**	**Binary**
0	000	0	_____
1	001	4	_____
2	010	2	_____
3	011	6	_____
4	100	1	_____
5	101	5	_____
6	110	3	_____
7	111	7	_____

To move an element from its original index to its correct position at the bottom of the recursion tree, we simply reverse the bits of its index! This is called a _____

The Butterfly Operation An Iterative FFT first applies the bit-reversal permutation to the array in $O(n)$ time. Then, it iteratively builds the results from the bottom up.

In each combine step, we update two elements u and v in place:

$$u_{new} = u + \omega \cdot v$$
$$v_{new} = u - \omega \cdot v$$

This fundamental constant-time update is known as a _____, named for the crossed-wing shape of its data-flow diagram. This allows the FFT to run in strictly $O(1)$ auxiliary space!

13 Real-World Applications

The FFT is frequently cited as the most important numerical algorithm of the 20th century. By breaking the $O(n^2)$ barrier for convolution, it enabled the digital revolution.

1. Digital Signal Processing (DSP) In engineering, signals (like audio) are recorded as a sequence of amplitudes over time (the Time Domain). The FFT converts this signal into the Frequency Domain, revealing the exact frequencies present in the sound.

- **Noise Filtering:** To remove static from a recording, one can take the FFT, zero-out the high-frequency components, and apply the Inverse FFT. Doing this naively takes $O(n^2)$, which is far too slow for real-time audio. FFT makes it possible.
- **Compression:** MP3 audio and JPEG images use variants of the FFT (like the Discrete Cosine Transform) to identify and discard frequencies that human ears and eyes cannot perceive, drastically reducing file sizes without noticeable quality loss.

2. Fast Integer Multiplication In modern cryptography (like RSA), computers must multiply prime numbers that are thousands of digits long. The grade-school multiplication algorithm takes $O(n^2)$ time.
However, we can treat a massive integer as a polynomial evaluated at $x = 10$! For instance:

$$1234 = 1(10^3) + 2(10^2) + 3(10^1) + 4(10^0)$$

The **Schonhage-Strassen Algorithm** converts huge numbers into polynomials, multiplies them using the FFT, and then processes the carries. This reduces large integer multiplication to _____ time, an essential speedup for encrypting data on the internet.